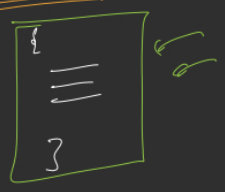
## Standard I/O



Types of I/O :

- Console I/O
- File I/O
- Network I/O ;
- Memory I/O !

## Console I/O :

```
System.out.println("Hello");
```

PrintStream

```
Class X {
    void println() {
        =
    }
}
```

```
Class System {
    static X out;
}
```

```
System.out.println("Hello");
```

```
System.out.println("Hello");
```

```
Class System {
}
```

→ java.lang

PrintStream

- println() → — new line
- print() → no new line
- printf() → formated o/P

```
Class System {
    static PrintStream out;
}

System s = new System();
s.out.println("Hello");
```

Input

↳ class System {

PrintStream out; // O/P

PrintStream err;

InputStream in;

}

System.in

↳ InputStream

Streams

↳ Flow of Data

InputStream → Data flow INTO program.

OutputStream → Data flow OUT of Program.

↳ Java Classes (Abstract)

read()

write()

InputStream → read();

OutputStream → write();

File Console N/W

System.in → InputStream

System.out, System.err → OutputStream

↳ PrintStream

Java I/O → Stream Based

Console

file

Memory

N/W

All of them are "Stream of Bytes"

(System.in) InputStream (Abstract) → read();

- ↳ FileInputStream → read() {}
- ↳ ByteArrayInputStream → read() {}
- ↳ BufferedInputStream → read() {}
- ↳ DataInputStream → read() {}

OutputStream (Abstract) → write();

- ↳ FileOutputStream → write() {}
- ↳ ByteArrayOutputStream → write() {}
- ↳ BufferedOutputStream → write() {}
- ↳ PrintStream → write() {}

(System.out)
(System.err)

System.in → InputStream
↳ Keyboard.

---

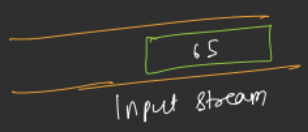
InputStream → read()
↳ Stream of Bytes

System.out.println();

System.in.read();

I/P: A → 65

ASCII




```
int data = System.in.read();
System.out.println((char) data);
```

read() → One byte at a time

A dity a → Input Buffer

A

Aditya → Buffer

[A] [d] [i] [t] [y] [a] [\n]

[65] [100] [105] [116] [121] [97] [10]

System.in.read()

// Aditya \n

```
String s = "";
int c;
while (c != 'n') {
    c = System.in.read();
    s += (char) c;
}
```

→ Cin>>

Scanf()

Reader (Abstract)

- → BufferedReader
- → InputStreamReader
- → FileReader.

Stream of characters

BufferedReader → Buffer

1 char
1 char
1 char
|
|
|

1000 OS Call

Keyboard → OS Buffer → Program

A → ~~DA~~ → AD

D

Keyboard → OS Buffer → Program

A → ~~DIA~~ → AD

D

Keyboard → OS Buffer → Java Buffer → Program

~~ILLY~~ ~~ILLY~~

ADITYA ADITYA

BufferedReader

- Read chunk of characters from OS Buffer.
- Store it in memory.
- Give them to program when required.

Standard I/P → InputStream

System.in

BufferedReader → Stream of char

InputStream → Stream of Bytes

InputStreamReader

## Input Stream Reader

byte Stream → character stream

InputStream

InputStreamReader (isr) = new ( System.in);

## Buffered Reader

BufferedReader (br) = new — (isr);

br.readLine();

### Limitations of Buffered Reader

String s = br.readLine();

int i = Integer.parseInt (s);

---

## JAVA - 1.5

↳ Scanner

↳ ① Simplify (Input)

↳ ② int, double, boolean, string

class Scanner {

=

}

→ java.io (X)

java.util

InputStream

↳

↳

↳

↳

Reader

↳

↳

↳

Scanner → Readable
↳ Reader

InputStream (System.in)
↑
InputStreamReader
↑
BufferedReader

## Creating Scanner

Keyboard → `Scanner sc = new Scanner( );`
System.in

File →
`Scanner sc = new Scanner( );`
new File("sample.txt")

String
`= new Scanner("10, 20, 30");`
sc.readLine();

## Scanner
↳ Tokenization

Hello | I | am | Aditya

## Methods of Scanner
↳ next()
nexline()
nextInt()